API for an External Encryption Service

Context

This document complements the Proposal to Apache Arrow to Extend PME and the Code-level External Encryptor + DLL interfaces/contracts documents. The PME extension considers the implementation of an external service that receives requests to encrypt/decrypt Arrow originated payloads as part of the new proposed architecture. This document describes the API of such a service.

Scope

This document aims to describe the API as a RESTful interface independent of a particular implementation. It describes two stateless endpoints that process a Parquet formatted payload for encryption/decryption and are meant to be functionally complete so that client users and implementations of the API server can complete a encryption-decryption roundtrip fully.

This API captures a minimal yet complete set of self-contained arguments required to encrypt/decrypt a Parquet dictionary page or data page (Parquet column chunks), independent of the larger Parquet file and column it belongs to.

The encrypt/decrypt RESTful calls:

(1) are fully contained to execute encrypts/decrypts without more context,

(2) have sufficient type information for individual data elements in the page to be parsed and can be type casted in runtime, and

(3) have application level context information.

This allows the API server implementation great flexibility to implement server-side targeted encryption at various levels. It could:

- Execute fully typed per-value encryption by physical datatype (INT32, FLOAT, BYTE_ARRAY, ..)
- Execute encryption by logical type (SSN, bank account numbers, customer PINs)
- Utilize the given application_context to have more sophisticated context-aware data protection and customization.
- Or simply encrypt the payload as a block of bytes in a more traditional encryption mode in a fine-tuned production server setup for encryption functionality.

POST /encrypt

Encrypts a single Parquet column chunk (Parquet dictionary page or data page). The request carries all the context needed to identify the data and describes the format of the payload. The response returns the encrypted data and the encryption metadata needed for future decryption.

Authentication

Bearer JWT token required in `Authorization` header.

The authentication flow of the service to obtain a JWT session token can be implemented in various ways, depending on the specific purpose. One option is for the API server to enable its own endpoint to receive and validate client credentials and generate its own JWTs.

However, other implementations can choose to delegate the authentication and generation of JWTs to a third-party service (like Auth0 or Clerk) or integration with a production Identity Provider (like Okta). Delegating authentication to a separate service could be preferable in many cases to allow the Encryption service implementing this API to remain as the logical functional encryption provider, and keep separation of responsibilities clear.

Request body

JSON

```
{
  "column_reference": {
    "name": "<string, required>"
  },
  "data_batch": {
    "datatype_info": {
      "datatype": "<string, required – see Datatype values section>",
      "length": "<integer, required only for FIXED_LEN_BYTE_ARRAY>"
    },
    "value": "<string, required – base64-encoded plaintext page payload>",
    "value_format": {
      "compression": "<string, required – see CompressionCodec values
section>",
      "encoding": "<string, required – see Encoding values section>",
      "encoding_attributes": {
        "<string>": "<string>", "– see Encoding_Attributes values section>"
      }
    }
  },
  "data_batch_encrypted": {
    "value_format": {
      "compression": "<string, required – compression to apply to the encrypted
output>"
    }
  },
  "encryption": {
    "key_id": "<string, required>"
  },
  "access": {
    "user_id": "<string, required>"
  },
  "application_context": "<string, required – JSON string with
application-level context>",
  "debug": {
    "reference_id": "<string, required – caller-provided ID for tracing>"
  }
}
```

Parquet page format

The **`data_batch`** section contains all the parameters to fully decode a Parquet dictionary or Parquet data page v1 or v2. For a fully contextualized description of Parquet formatting, see the official Apache Parquet documentation, here: Data Pages | Parquet

Top-level fields

`column_reference`: Identifies the column being encrypted. Contains the column name as it appears in the Parquet schema. This could be used by the server for key selection, logging, or audit purposes.

`data_batch`: Carries the plaintext payload and all the metadata needed to interpret it. Contains three sub-sections:

- `datatype_info` describes the physical type of the column values (e.g. INT32, BYTE_ARRAY) and, for fixed-length types, the byte width
- `value` holds the actual page data as a base64-encoded binary blob, and
- `value_format` specifies the compression and encoding applied to that data, along with the encoding-specific attributes needed to properly parse it (such as page type, number of values, and level metadata).

`data_batch_encrypted`: Describes the desired format of the encrypted output. Currently contains only `value_format.compression`, which specifies the compression codec to apply to the ciphertext before returning it. This allows the caller to control the compression of the encrypted payload independently from the input compression.

`encryption`: Contains the `key_id` identifying which encryption key the server should use. This is a logical identifier, not the key material itself. The server resolves it to the actual key internally, so the data and the key to encrypt it don't travel together on the request.

`access`: Carries the `user_id` of the caller. This is forwarded to the server for access control decisions and audit logging. The server uses it to determine the role and whether access should be granted.

`application_context`: A free-form JSON string provided by the calling application. Passed through to the server as additional context from application to server. It's an application-server contract. Can be used for policy evaluation or specialized context-aware encryption.

`debug`: Contains a `reference_id` generated by the client for request tracing and debugging purposes.

Enum values (datatype, compression, encoding)

`data_batch.datatype_info.datatype`
— physical data type of the column values:

Value	Description
`BOOLEAN`	Boolean
`INT32`	32-bit integer
`INT64`	64-bit integer
`INT96`	96-bit integer (legacy Parquet timestamp)
`FLOAT`	32-bit IEEE float
`DOUBLE`	64-bit IEEE double
`BYTE_ARRAY`	Variable-length byte array
`FIXED_LEN_BYTE_ARRAY`	Fixed-length byte array (requires `datatype_info.length`)

`data_batch.value_format.compression` / `data_batch_encrypted.value_format.compression`
— compression codec applied to the data:

Value	Description
`UNCOMPRESSED`	No compression
`SNAPPY`	Snappy
`GZIP`	GZIP
`BROTLI`	Brotli
`ZSTD`	Zstandard
`LZ4`	LZ4 raw
`LZ4_FRAME`	LZ4 framed
`LZO`	LZO
`BZ2`	BZip2

LZ4_HADOOP	LZ4 Hadoop-compatible variant

data_batch.value_format.encoding
— encoding applied to the column page values:

Value	Description
PLAIN	Plain encoding
RLE	Run-length encoding / bit-packing hybrid
BIT_PACKED	Bit-packed encoding (deprecated)
DELTA_BINARY_PACKED	Delta encoding for integers
DELTA_LENGTH_BYTE_ARRAY	Delta encoding for byte array lengths
DELTA_BYTE_ARRAY	Delta encoding for byte arrays
RLE_DICTIONARY	RLE dictionary encoding
BYTE_STREAM_SPLIT	Byte stream split for floating point

Encoding attributes map

encoding_attributes
— map of string key-value pairs. The required keys depend on the value of page_type.

Always required:

Key	Type	Description
page_type	string	The Parquet page type. One of: DATA_PAGE_V1, DATA_PAGE_V2, DICTIONARY_PAGE

When page_type is DATA_PAGE_V1 or DATA_PAGE_V2:

Key	Type	Description
data_page_num_values	integer (as string)	Number of values in the data page (including nulls)
data_page_max_definition_level	integer (as string)	Max definition level from the column schema
data_page_max_repetition_level	integer (as string)	Max repetition level from the column schema

When page_type is DATA_PAGE_V1 (in addition to the common data page attributes):

Key	Type	Description
`page_v1_definition_level_encoding`	string (excepted “RLE”)	Encoding used for definition levels
`page_v1_repetition_level_encoding`	string (excepted “RLE”)	Encoding used for repetition levels

When page_type is DATA_PAGE_V2 (in addition to the common data page attributes):

Key	Type	Description
`page_v2_definition_levels_byte_length`	integer (as string)	Byte length of the definition levels section
`page_v2_repetition_levels_byte_length`	integer (as string)	Byte length of the repetition levels section
`page_v2_num_nulls`	integer (as string)	Number of null values in the page
`page_v2_is_compressed`	boolean (as string)	Whether the data section is compressed. "true" or "false"

When page_type is DICTIONARY_PAGE:

Key	Type	Description
`dict_page_num_values`	integer (as string)	Number of entries in the dictionary

All values are passed as strings within the JSON map. Integer and boolean values must be parseable from their string representation (e.g. "42", "true").

Response body (`200` OK)

JSON

```
{
  "data_batch_encrypted": {
    "value": "<string — base64-encoded ciphertext>",
    "value_format": {
      "compression": "<string, required — see CompressionCodec values section>"
    }
  },
  "encryption_metadata": {
    "<string>": "<string>", "— see Encryption metadata map section>"
  },
```

```
  "access": {
    "user_id": "<string — echoed from request>",
    "role": "<string — role determined by access control>",
    "access_control": "<string — e.g. granted>"
  },
  "debug": {
    "reference_id": "<string — echoed from request>"
  }
}
```

Encryption metadata map

The `encryption_metadata` map must be persisted by the caller. It is required as input to the corresponding `/decrypt` call.

Keys:

Key	Values	Description
`dbps_agent_version`	string	Version of the DBPS encryption logic used. Validated at decryption time to ensure compatibility.
`encrypt_mode`	`per_value or per_block`	Encryption mode used for all data pages or dictionary pages.

Encryption mode values:

Value	Meaning
`per_value`	Each value in the page was individually encrypted.
`per_block`	The entire page payload was encrypted as a single opaque block. Used as a fallback when per-value encryption is not supported for the given combination of encoding, compression, or datatype.

Error responses

Status	Condition
`400`	Missing or invalid request fields; body contains error with detail
`401`	Missing, invalid, or expired JWT token
`400`	Encryption processing failure; body contains error with stage and detail

POST /decrypt

Decrypts a single Parquet column page. The request carries the ciphertext, all the context needed to identify the data and its original format, and the encryption metadata that was produced during the corresponding `/encrypt` call. The response returns the recovered plaintext along with the original data type and format information.

Authentication is identical as the `/encrypt` call.

Request body

```
JSON
{
  "column_reference": {
    "name": "<string, required>"
  },
  "data_batch": {
    "datatype_info": {
      "datatype": "<string, required – see Type values section>",
      "length": "<integer, required only for FIXED_LEN_BYTE_ARRAY>"
    },
    "value_format": {
      "compression": "<string, required – see CompressionCodec values
section>",
      "encoding": "<string, required – see Encoding values section>",
      "encoding_attributes": {
        "<string>": "<string>", "– see Encoding_Attributes values section>"
      }
    }
```

```
  },
  "data_batch_encrypted": {
    "value": "<string, required – base64-encoded ciphertext>",
    "value_format": {
      "compression": "<string, required – compression applied to the encrypted
payload>"
    }
  },
  "encryption": {
    "key_id": "<string, required>"
  },
  "encryption_metadata": {
    "<string>": "<string>", "– see Encryption metadata map section>"
  },
  "access": {
    "user_id": "<string, required>"
  },
  "application_context": "<string, required – JSON string with
application-level context>",
  "debug": {
    "reference_id": "<string, required – caller-provided ID for tracing>"
  }
}
```

Structurally the `/encrypt` and `/decrypt` request bodies are identical except in the following:

- Where the payload is specified:
  - In `/encrypt`, the payload (plaintext) is in `data_batch.value`.
  - In `/decrypt`, the payload (ciphertext) is in `data_batch_encrypted.value`.

- encryption_metadata:
  - Present only in `/decrypt`. This is the map returned by the server in the `/encrypt` response, passed back to `/decrypt` so the server knows how to reverse the encryption (version, encryption mode).

Response body (`200 OK`)

```JSON
{
  "data_batch": {
    "datatype_info": {
      "datatype": "<string – echoed from request>",
      "length": "<integer – echoed from request, if applicable>"
    },
    "value": "<string – base64-encoded recovered plaintext>",
    "value_format": {
      "compression": "<string – echoed from request>",
      "encoding": "<string – echoed from request>"
    }
  },
  "access": {
    "user_id": "<string – echoed from request>",
    "role": "<string – role determined by access control>",
    "access_control": "<string – e.g. granted>"
  },
  "debug": {
    "reference_id": "<string – echoed from request>"
  }
}
```

Structurally, in the `/decrypt` response:

- The `data_batch` section is identical to data_batch from the `/decrypt` request, but with the addition of `value`, which contains the base64-encoded recovered plaintext. The `datatype_info` and `value_format` fields are copied back from the request to the response.

- The `access` and `debug` sections are identical to the `/encrypt` response.

Error responses

Status	Condition
`400`	Missing or invalid request fields; body contains error with detail
`401`	Missing, invalid, or expired JWT token

`400`	Encryption processing failure; body contains error with stage and detail

Swagger yaml and examples

See Swagger source

- https://github.com/protegrity/DataBatchProtectionService/blob/main/src/common/swagger.yaml

Swagger rendered

- https://redocly.github.io/redoc/?url=https://raw.githubusercontent.com/protegrity/DataBatchProtectionService/main/src/common/swagger.yaml
- https://petstore.swagger.io/?url=https://raw.githubusercontent.com/protegrity/DataBatchProtectionService/main/src/common/swagger.yaml

<>